BRAC UNIVERSITY

Inspiring Excellence

CSE471: System Analysis and Design
Project Report
Project Title:

Ekotro (একত্র) - Bringing Governance Together

Item	Details
Course	CSE471: System Analysis and Design
Group No.	01
Lab Section	15
Semester	Summer 2026
Submission Date	6 September, 2026
GitHub Repository	https://github.com/rifff-fin/471gg
Live Frontend	https://ekotro.netlify.app/
Backend	https://ekotrobackend.onrender.com/

Team members:

ID	Name
24141141	Ariful Islam Naeem
23301214	Samara Shahjeen Huq
22201979	Abdullah Al Rifat
23101321	S M Sabbir Haque Emon (Dropped the course)

Submission Date:

6, September, 2026

Table of Contents

1. System Request

Business need: Citizens often have no single, transparent channel for reporting local problems, following progress, communicating with officials, requesting government services, or checking enforcement actions. Government teams also need a shared system to prioritize cases, assign field workers, exchange updates, verify completed work, and monitor service performance.

Ekotro addresses this gap with a unified civic governance platform. It connects citizens, ward-level authorities, officers, police officers, administrators, and field workers around a common complaint and service workflow.

Business requirements:

The system shall:

- Let authenticated citizens submit civic complaints with title, category, description, ward, location, optional images, and priority information.
- Make public complaints searchable and filterable by category, status, date, priority, and location.
- Allow citizens to edit or delete their own unassigned complaints.
- Provide map-based complaint discovery using Leaflet and MongoDB geospatial queries.
- Support public comments, threaded replies, votes, and real-time complaint activity.
- Let officials publish announcements and responses.
- Let officers review cases, assign field workers, place cases on hold, release held cases, and maintain a public ledger.
- Let field workers submit completion reports with before-and-after evidence.
- Provide unified government service requests for passports, driving licenses, birth certificates, and similar services.
- Let police officers issue digital fines and let citizens view and track their fines.
- Provide role-based dashboards, notifications, analytics, SLA monitoring, and hotspot information.

Business value:

- Citizens: one portal for reporting issues, requesting services, communicating, and tracking outcomes.
- Government officers: structured case queues, role-based access, assignment tools, public audit history, and analytics.

- Field crews: direct task assignments, internal coordination, progress updates, and evidence submission.
- Administrators: visibility into complaint volume, SLA breaches, hotspots, department activity, and engagement.
- Community: greater transparency because public updates, votes, status changes, and ledger events are visible.

Special issues or constraints:

- Users require an internet connection and a modern browser.
- MongoDB, Cloudinary, Render, Netlify, OpenStreetMap tiles, and Socket.IO are external service dependencies.
- Cloudinary credentials must be configured on the backend before image uploads can succeed.
- The backend may sleep on the Render free tier, causing the first request to take longer.
- Location access depends on browser permissions; users can also enter latitude and longitude manually.
- Authentication and authorization depend on valid JWT configuration and role data.
- Uploaded files are restricted by the server upload middleware to image files, with a maximum of eight complaint attachments and twelve megabytes per file.
- The report must include original screenshots from the deployed system and tools. Placeholder entries are marked below and should be replaced before submission.

2. Functional Requirements

No.	Functional requirement	Assigned member	Implemented evidence
1	Citizens submit complaints with title, category, description, location, and priority	Abdullah Al Rifat	ReportIssue.jsx, complaintRoutes.js, Complaint.js
2	Citizens browse, search, and filter public complaints	Samara Shahjeen Huq	Feed.jsx, public complaint API
3	Cloudinary handles complaint image hosting, compression, and delivery	S M Sabbir Haque Emon	services/cloudinary.js, upload middleware
4	Citizens edit, update, or delete their own unassigned complaints	Ariful Islam Naeem	updateComplaint, deleteComplaint, edit page

No.	Functional requirement	Assigned member	Implemented evidence
5	Leaflet and MongoDB geospatial indexing support complaint maps and nearby queries	Ariful Islam Naeem	MapPicker.jsx, Complaint.js, /nearby route
6	Citizens comment on complaints and reply to comments	Samara Shahjeen Huq	comment and reply endpoints, complaint detail page
7	Socket.IO supports real-time public complaint activity	Abdullah Al Rifat	services/realtime.js, notification components
8	Citizens upvote or downvote existing complaints	S M Sabbir Haque Emon	/upvote and /vote endpoints
9	Councillors and mayors publish official responses, updates, and announcements	Abdullah Al Rifat	announcement routes, mayor dashboard
10	Field workers upload completion reports with before-and-after images	Ariful Islam Naeem	completion report routes and upload page
11	Cloudinary processes before-and-after field evidence	Abdullah Al Rifat	toMediaList, Cloudinary upload stream
12	Socket.IO provides internal officer and crew coordination	S M Sabbir Haque Emon	coordination rooms and events in realtime.js
13	Citizens submit and track government service requests	Samara Shahjeen Huq	service request routes and government services page
14	Officers review, approve, reject requests, and assign officer roles	Abdullah Al Rifat	officer routes, service request controller, admin dashboard
15	Officers can put complex complaints into HELD_PENDING with a mandatory rationale and ledger event	S M Sabbir Haque Emon	hold/release endpoints and publicLedger model field
16	Police officers issue digital fines with evidence	Ariful Islam Naeem	fine routes, fineController.js, police dashboard

No.	Functional requirement	Assigned member	Implemented evidence
17	Citizens view fines and follow review/dispute information	Samara Shahjeen Huq	My Fines page and fine API
18	Administrators monitor analytics, performance, engagement, requests, and activity	Abdullah Al Rifat	admin stream and analytics endpoints/dashboard

Main User Roles

- **Citizen:** Reports issues, comments, votes, requests services, views fines, and receives updates.
- **Officer:** Reviews complaints and service requests, assigns crews, verifies reports, and coordinates work.
- **Councillor / Mayor:** Publishes official responses and announcements and manages jurisdictional updates.
- **Field Worker:** Receives maintenance assignments, posts progress, and uploads completion evidence.
- **Police Officer:** Issues and views digital fines.
- **Administrator:** Manages users and system-wide operational dashboards.

3. Technology (Framework, Languages)

Layer	Implemented technology
Frontend language	JavaScript / JSX
Frontend framework	React 19
Frontend build tool	Vite
Routing	React Router
Styling	Project CSS (App.css, index.css)
Backend language	JavaScript, CommonJS
Backend framework	Express 5
Database	MongoDB
ODM	Mongoose
Authentication	JWT and bcryptjs
File upload	Multer
Media platform	Cloudinary
Maps	Leaflet and React Leaflet with OpenStreetMap tiles

Layer	Implemented technology
Real-time communication	Socket.IO and socket.io-client
Frontend hosting	Netlify
Backend hosting	Render

High-Level Architecture

```
flowchart LR
  C[Citizen / Officer / Crew Browser] -->|HTTPS| F[React + Vite Frontend on Netlify]
  F -->|REST JSON and multipart| B[Express API on Render]
  F <-->|Socket.IO events| B
  B -->|Mongoose| M[(MongoDB)]
  B -->|Image upload stream| CL[Cloudinary]
  F -->|Map tiles| O[OpenStreetMap]
```

4. Backend Development

The backend is an Express API organized into routes, controllers, models, middleware, and services. JWT middleware protects private routes, while role authorization limits operations to the correct actor.

Backend Snippet 1: Express and Socket.IO Server Setup

File: server.js

```
const app = express();
const httpServer = http.createServer(app);

const io = new Server(httpServer, {
  cors: {
    origin: process.env.CLIENT_ORIGIN || "*",
    methods: ["GET", "POST", "PUT", "PATCH", "DELETE"],
  },
});

app.set("io", io);
registerSocketHandlers(io);
app.use(express.json());
app.use(express.urlencoded({ extended: true, limit: "10mb" }));

app.use("/api/auth", authRoutes);
app.use("/api/complaints", complaintRoutes);
app.use("/api/fines", fineRoutes);
app.use("/api/service-requests", serviceRequestRoutes);
```

Description: Creates the HTTP server, attaches Socket.IO, enables JSON and form parsing, and mounts the main feature APIs.

Backend Snippet 2: Protected Complaint Routes

File: routes/complaintRoutes.js

```
router.post(
  "/",
  protect,
  authorize("citizen"),
  upload.array("attachments", 8),
  createComplaint,
);

router.put("/:id", protect, authorize("citizen"), updateComplaint);
router.delete("/:id", protect, authorize("citizen"), deleteComplaint);

router.post(
  "/:id/reports",
  protect,
  authorize("field_worker", "officer", "admin"),
  upload.fields([
    { name: "beforeImages", maxCount: 5 },
    { name: "afterImages", maxCount: 5 },
  ]),
  addCompletionReport,
);
```

Description: Demonstrates authentication, role authorization, and separate upload rules for citizen complaints and field-worker completion reports.

Backend Snippet 3: JWT Authentication and Role Authorization

File: middleware/authMiddleware.js

```
const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({
      success: false,
      message: "Authentication required",
    });
  }

  const token = authHeader.split(" ")[1];
  req.user = jwt.verify(token, JWT_SECRET);
  next();
};

const authorize = (...roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role)) {
    return res.status(403).json({
      success: false,
      message: `Access denied. Required role: ${roles.join(" or ")}`,
    });
  }
```

```
  next();
};
```

Description: Verifies the bearer token and prevents users from accessing routes outside their assigned role.

Backend Snippet 4: Cloudinary Image Upload and Compression

File: services/cloudinary.js

```
const uploadBuffer = (buffer, options = {}) => {
  if (!buffer || !buffer.length) {
    return Promise.reject(new Error("No upload buffer provided."));
  }

  return new Promise((resolve, reject) => {
    const stream = cloudinary.uploader.upload_stream(
      {
        folder: options.folder || "ekotro",
        resource_type: options.resource_type || "image",
        transformation: options.transformation || [
          { quality: "auto:good" },
        ],
        unique_filename: true,
      },
      (error, result) => (error ? reject(error) : resolve(result)),
    );

    const readable = new PassThrough();
    readable.end(buffer);
    readable.pipe(stream);
  });
};
```

Description: Sends the in-memory upload stream directly to Cloudinary, applies automatic image quality optimization, and returns the hosted image result without writing files to the server disk.

Backend Snippet 5: Geospatial Complaint Model

File: models/Complaint.js

```
location: {
  type: { type: String, enum: ["Point"], default: "Point" },
  coordinates: { type: [Number], required: true },
},

complaintSchema.index({ location: "2dsphere" });
```

Description: Stores locations in GeoJSON Point format as [longitude, latitude] and creates a MongoDB 2dsphere index for nearby and spatial complaint queries.

Backend Snippet 6: Complaint Location Normalization

File: controllers/complaintController.js

```
const normalizeLocation = (body = {}) => {
  const rawLat = body.latitude ?? body.lat ?? body.locationLat;
  const rawLng = body.longitude ?? body.lng ?? body.locationLng;
  const lat = Number(rawLat);
  const lng = Number(rawLng);

  if (
    Number.isFinite(lat) &&
    Number.isFinite(lng) &&
    Math.abs(lat) <= 90 &&
    Math.abs(lng) <= 180
  ) {
    return { type: "Point", coordinates: [lng, lat] };
  }

  return null;
};
```

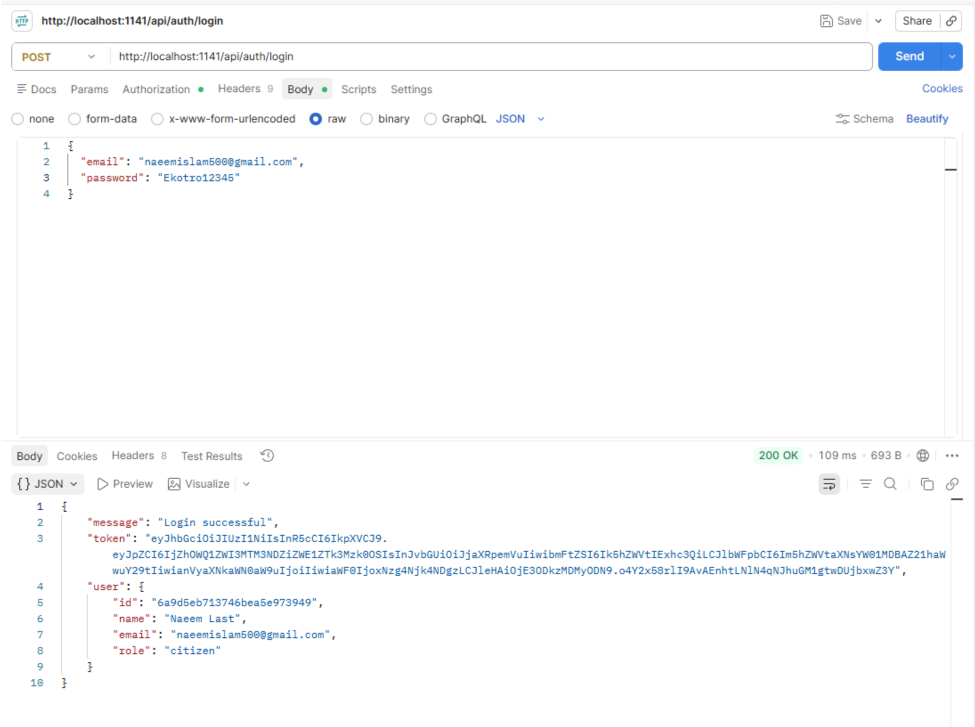
Description: Accepts common latitude and longitude field names, validates coordinate ranges, and converts them to the GeoJSON structure used by MongoDB.

Backend API Summary

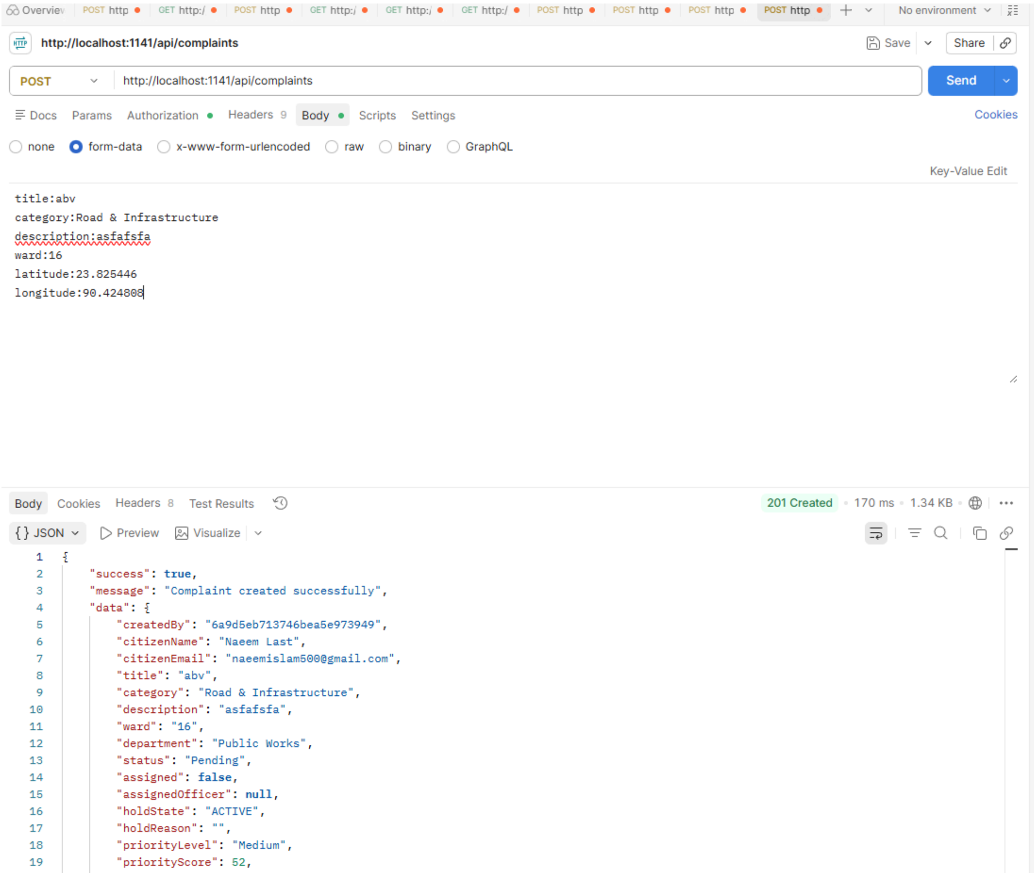
Feature	Method and path	Access
Public complaint feed	GET /api/complaints	Public
Nearby complaints	GET /api/complaints/nearby	Public
Create complaint	POST /api/complaints	Citizen
Edit/delete own complaint	PUT/DELETE /api/complaints/:id	Citizen owner
Comment and reply	POST /api/complaints/:id/comments	Authenticated
Upvote/vote	POST /api/complaints/:id/upvote or /vote	Authenticated
Hold/release complaint	POST /api/complaints/:id/hold or /release	Officer, councillor, mayor, admin
Completion report	POST /api/complaints/:id/reports	Field worker, officer, admin
Service request	POST /api/service-requests	Citizen
Issue fine	POST /api/fines	Police
Citizen fines	GET /api/fines/my	Citizen
Admin analytics	GET /api/complaints/admin/analytics	Admin/officer

Postman Evidence Checklist

- **[Figure: Screenshot of Postman 1]** POST /api/auth/login returns a JWT token.

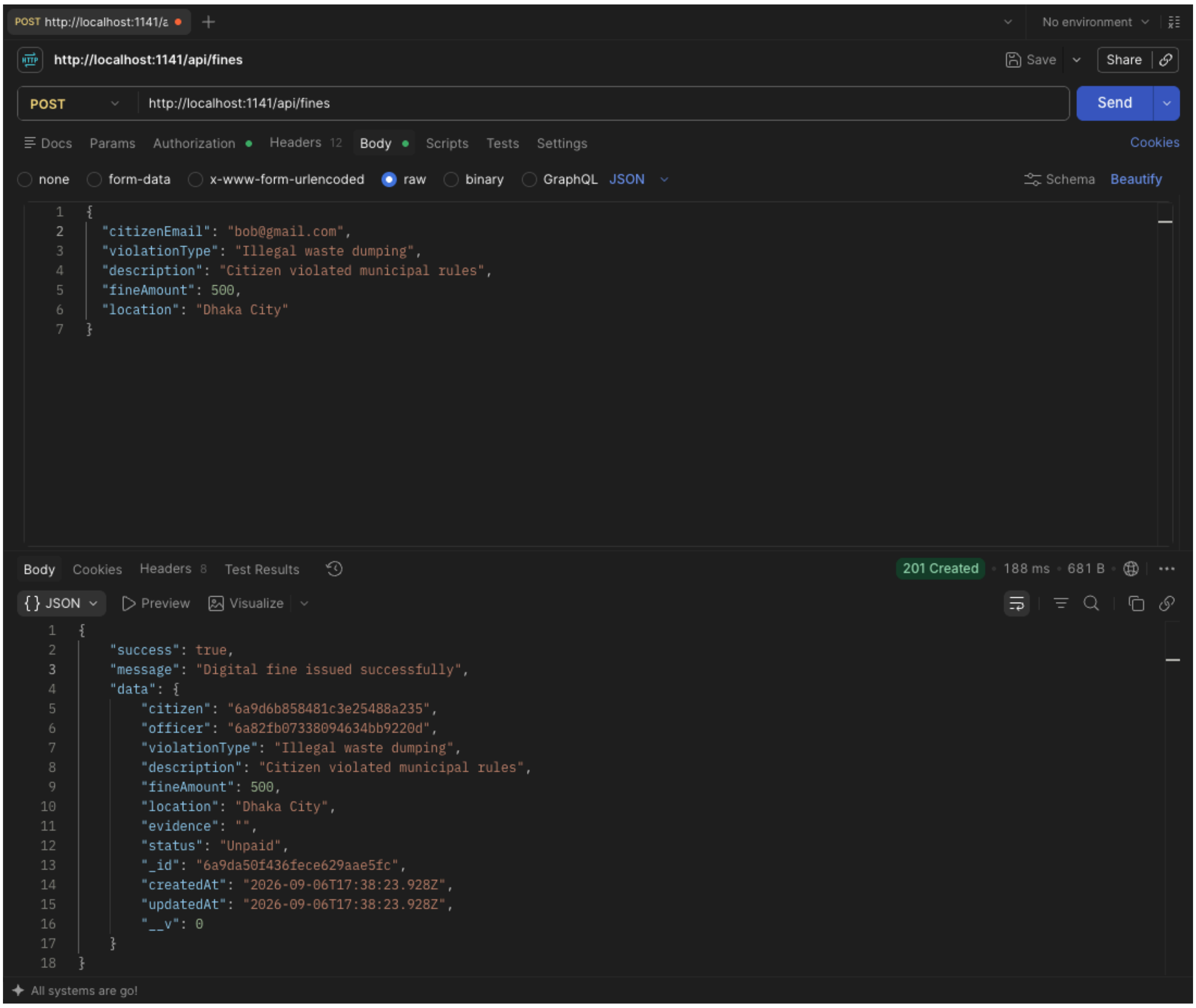


- **[Figure: Screenshot of Postman 2]** POST /api/complaints creates a complaint with multipart fields and an attachment.

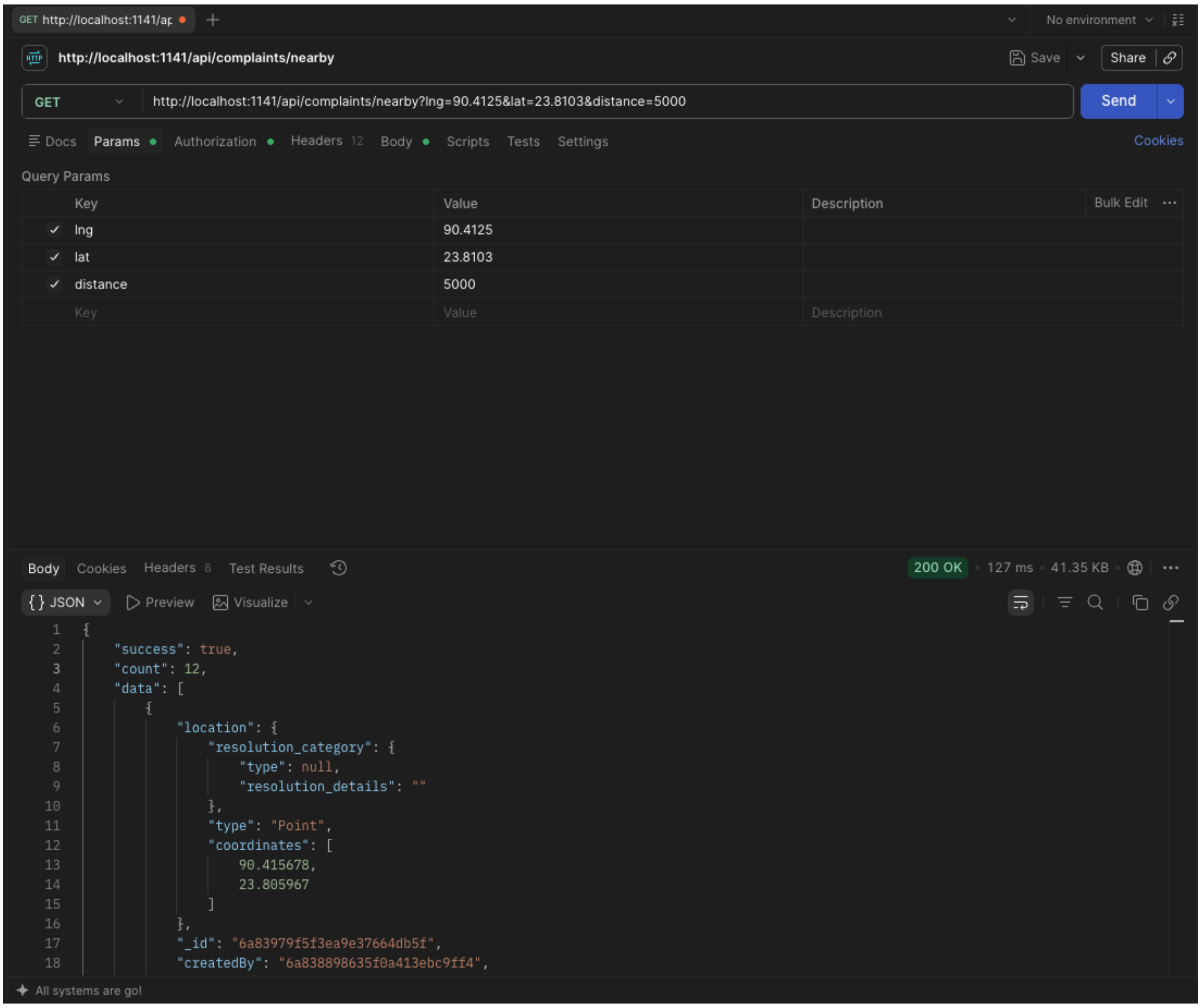


- **[Figure: Screenshot of Postman 3]** POST /api/complaints/:id/hold returns a held complaint and records the reason.

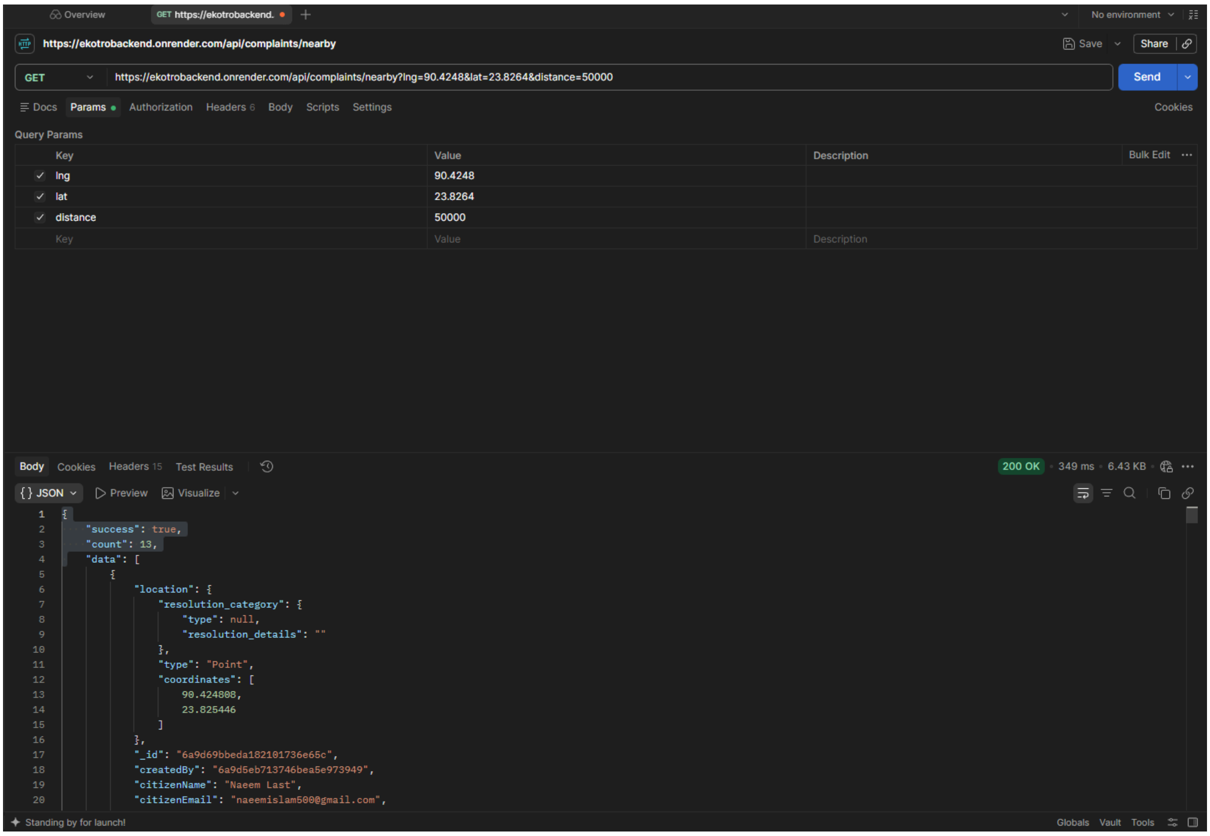
- **[Figure: Screenshot of Postman 4]** POST /api/fines creates a police digital fine.

- 

- **[Figure: Screenshot of Postman 5]** GET /api/complaints/nearby returns location-filtered results.



- **[Figure: Screenshot of Postman 6]** — GET /api/complaints/nearby



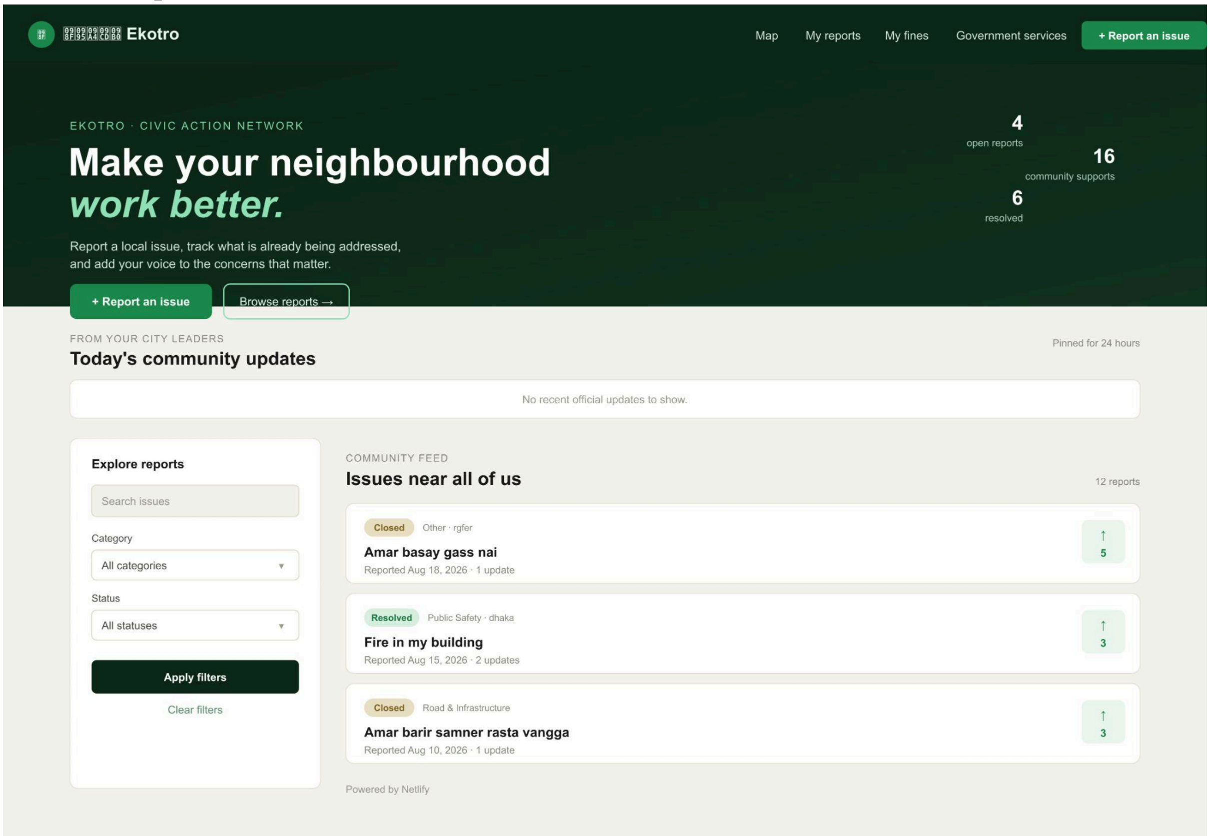
5. User Interface Design

The interface uses a civic-service visual language with a persistent navigation bar, role-specific dashboards, responsive forms, complaint cards, status indicators, map views, notifications, and focused detail pages.

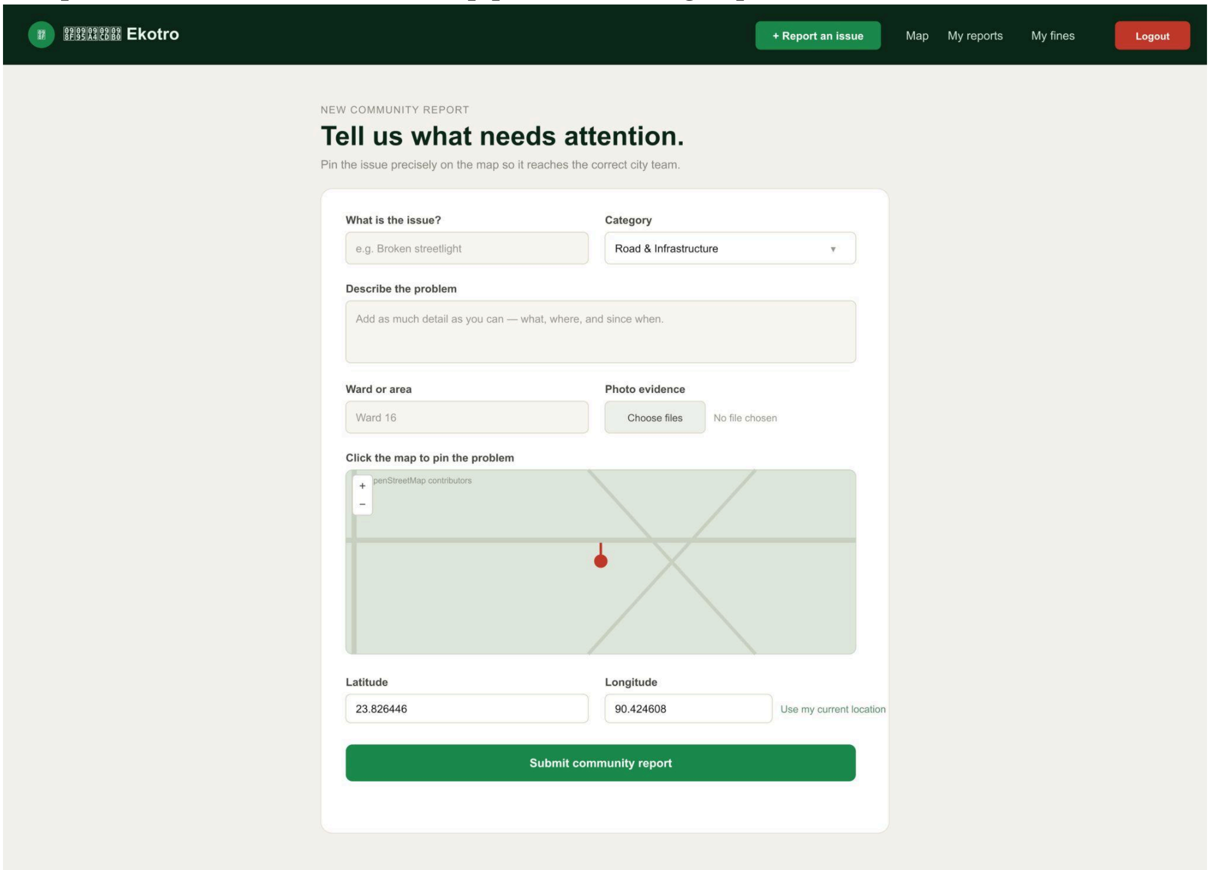
Required Figma Evidence

The following five designs should be exported from the team Figma file as PNG/JPG images and inserted into this section:

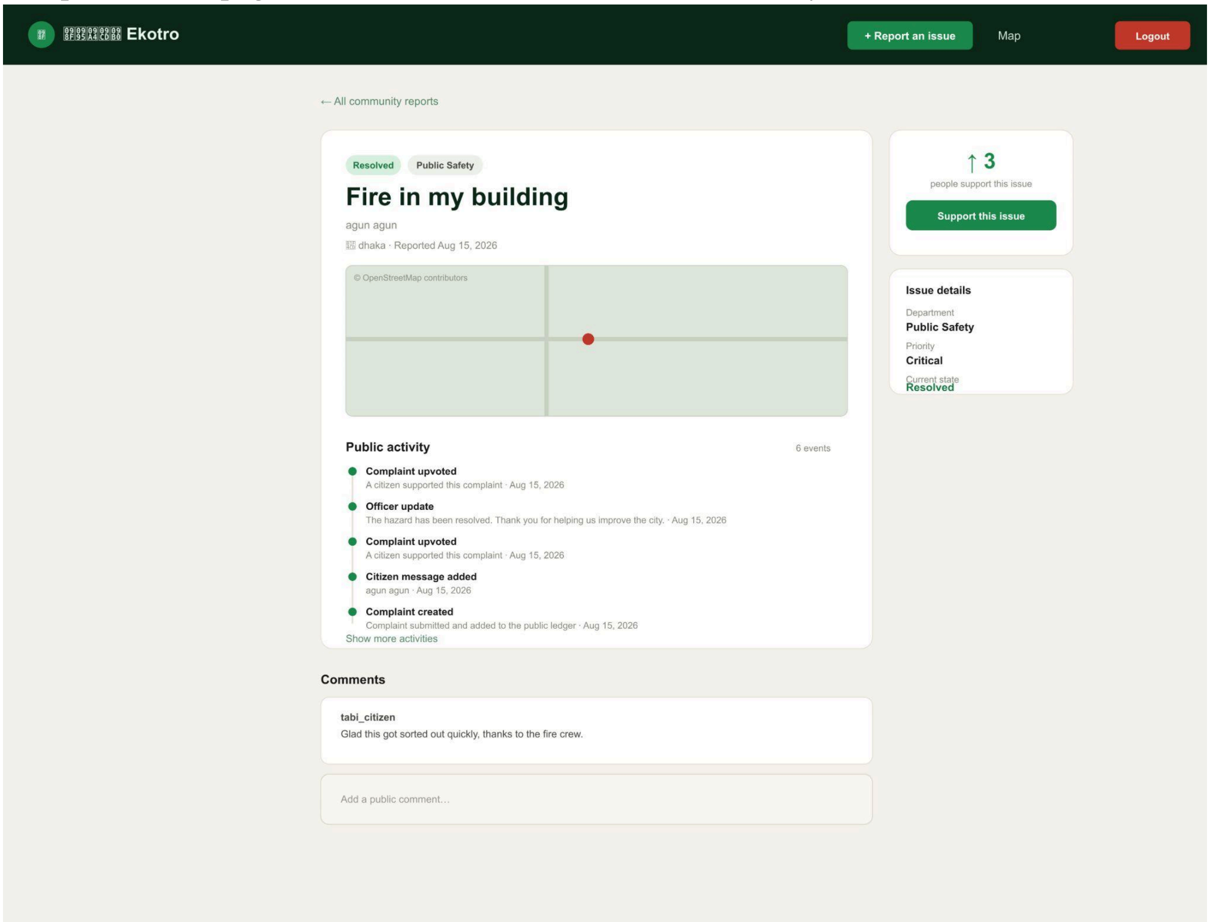
1. Citizen complaint feed with search and filters.



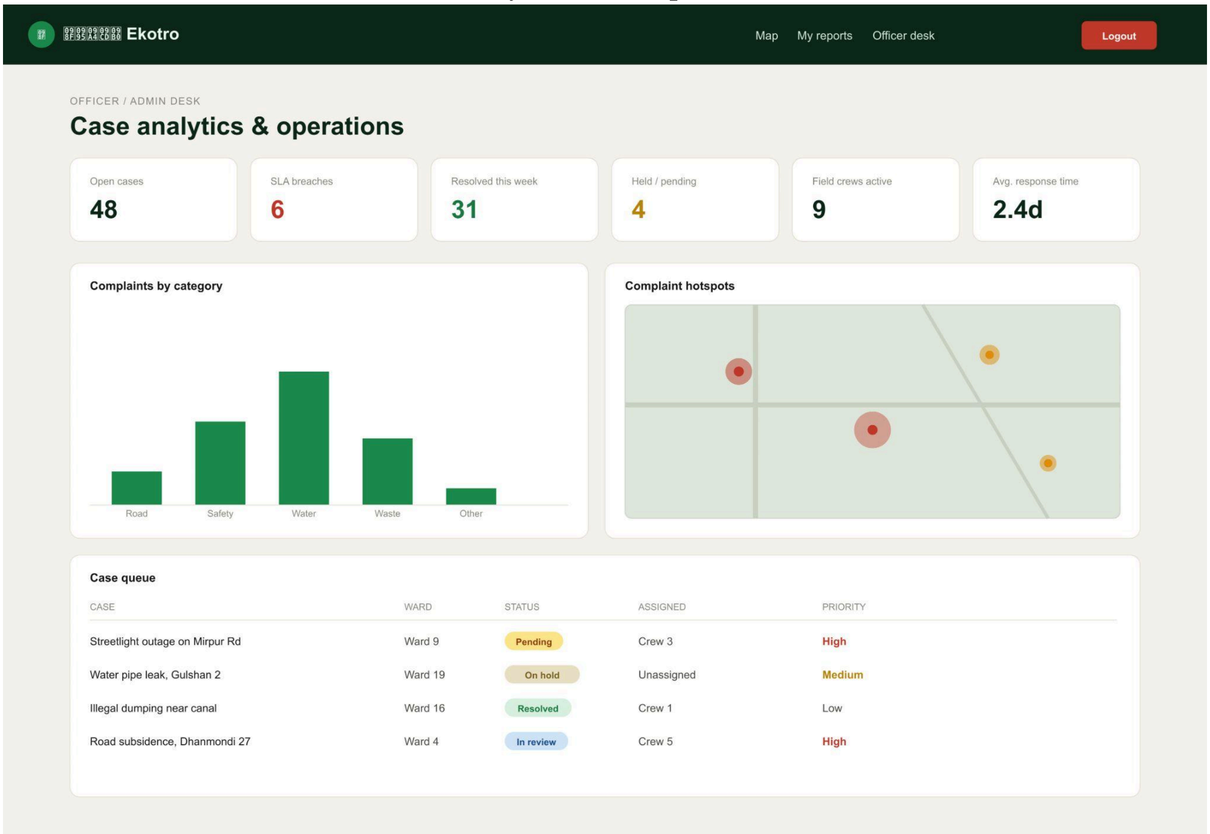
2. Complaint submission form with map picker and image upload.



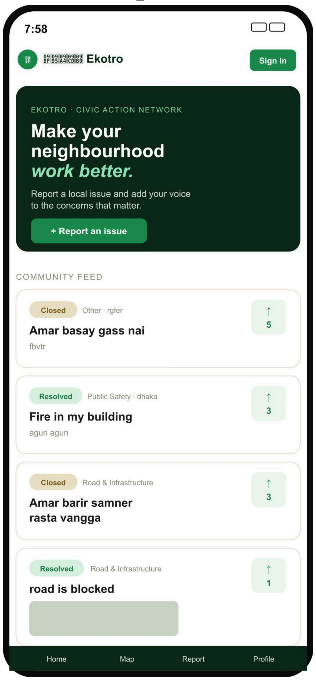
3. Complaint detail page with status, votes, comments, and activity.



4. Officer/admin dashboard with case analytics and map data.



5. Mobile responsive view of the complaint or service workflow.



Figma project link: [Figma Link]

The GitHub repository and deployed URLs are available above, but no Figma URL was provided in the project request. Add the actual public Figma link before submission.

6. Frontend Development

The frontend is a React single-page application. React Router maps user roles to protected pages, Axios provides the API client, Leaflet handles map interaction, and Socket.IO displays live notifications.

Frontend Snippet 1: Axios API Client and JWT Header

File: `frontend/src/services/api.js`

```
export const API_BASE_URL =
  import.meta.env.VITE_API_BASE_URL || "http://localhost:1141/api";
export const SOCKET_URL =
  import.meta.env.VITE_SOCKET_URL || API_BASE_URL.replace(/\/api\/?$/,
"");

const api = axios.create({ baseURL: API_BASE_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
```

```
  return config;
});
```

Description: Uses environment configuration for deployment and automatically adds the stored JWT to authenticated API requests.

Frontend Snippet 2: Complaint Submission with FormData

File: `frontend/src/pages/ReportIssue.jsx`

```
const submit = async (event) => {
  event.preventDefault();
  setSaving(true);
  setError("");
  try {
    const data = new FormData();
    Object.entries(form).forEach(([key, value]) => data.append(key,
value));
    files.forEach((file) => data.append("attachments", file));
    const response = await api.post("/complaints", data);
    navigate(`/complaints/${response.data.data._id}`);
  } catch (requestError) {
    setError(
      requestError.response?.data?.message || "Could not submit your
report.",
    );
  } finally {
    setSaving(false);
  }
};
```

Description: Collects form values and optional image files in a multipart request and routes the user to the newly created complaint.

Frontend Snippet 3: Leaflet Location Picker

File: `frontend/src/components/MapPicker.jsx`

```
function LocationMarker({ onLocationSelect }) {
  const [position, setPosition] = useState(null);

  useMapEvents({
    click(e) {
      const location = { lat: e.latlng.lat, lng: e.latlng.lng };
      setPosition(location);
      onLocationSelect(location);
    },
  });

  return position ? <Marker position={[position.lat, position.lng]} /> :
null;
}
```

Description: Converts a map click into latitude and longitude values and displays a marker at the selected location.

Frontend Snippet 4: Role-Based Route Protection

File: `frontend/src/App.jsx`

```
<Route
  path="/officer-dashboard"
  element={
    <RoleProtectedRoute allowedRoles={["officer", "admin"]}>
      <OfficerDashboard />
    </RoleProtectedRoute>
  }
/>

<Route
  path="/police/create-fine"
  element={
    <RoleProtectedRoute allowedRoles={["police"]}>
      <CreateFine />
    </RoleProtectedRoute>
  }
/>
```

Description: Shows how the client hides and protects pages based on the authenticated user role. The backend independently enforces the same authorization rules.

Frontend Snippet 5: Real-Time Notifications

File: `frontend/src/components/NotificationCenter.jsx`

```
useEffect(() => {
  if (!user || !token) return undefined;
  const socket = io(SOCKET_URL, {
    transports: ["websocket", "polling"],
    auth: { token },
  });

  socket.on("notification:new", (payload) => {
    window.dispatchEvent(new Event("notification:received"));
    setNotice({
      message: payload.message || payload.title || "You have a new case 
update.",
    });
  });

  return () => socket.disconnect();
}, [user, token]);
```

Description: Opens a token-authenticated Socket.IO connection, shows live case notifications, and disconnects cleanly when the user session changes.

7. User Manual

7.1 Register and Log In

1. Open https://ekotro.netlify.app/.
2. Select **Register** and provide a name, email, and password.
3. Log in with the registered credentials.
4. The system stores the JWT session and displays features allowed for the user role.

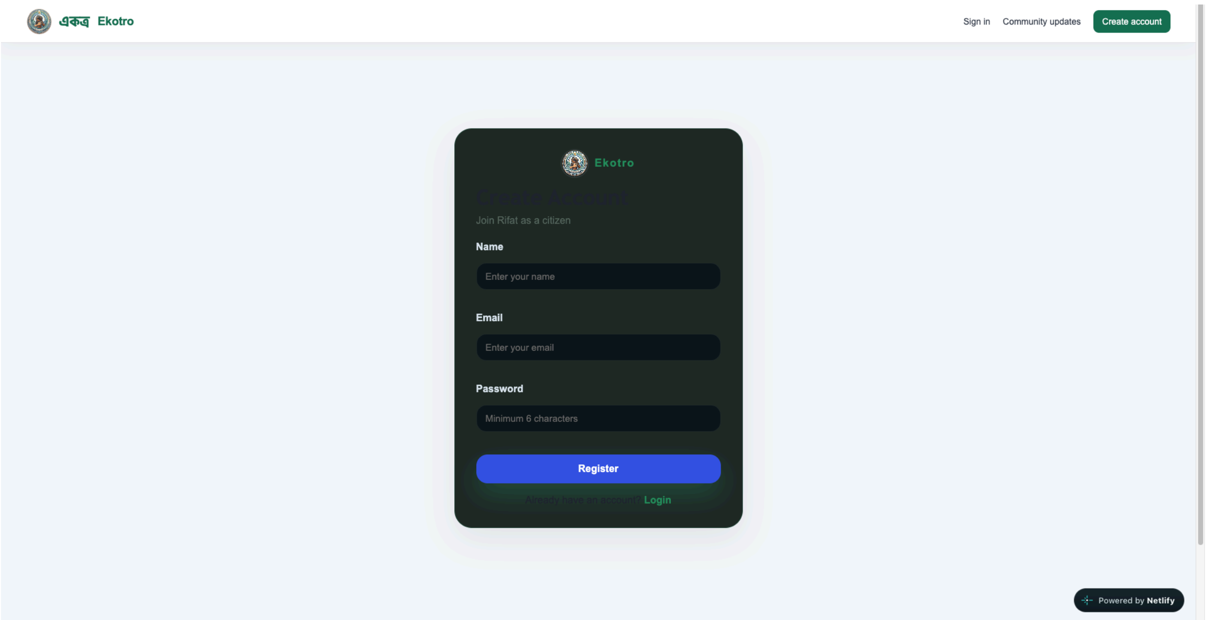


Figure: Registration page

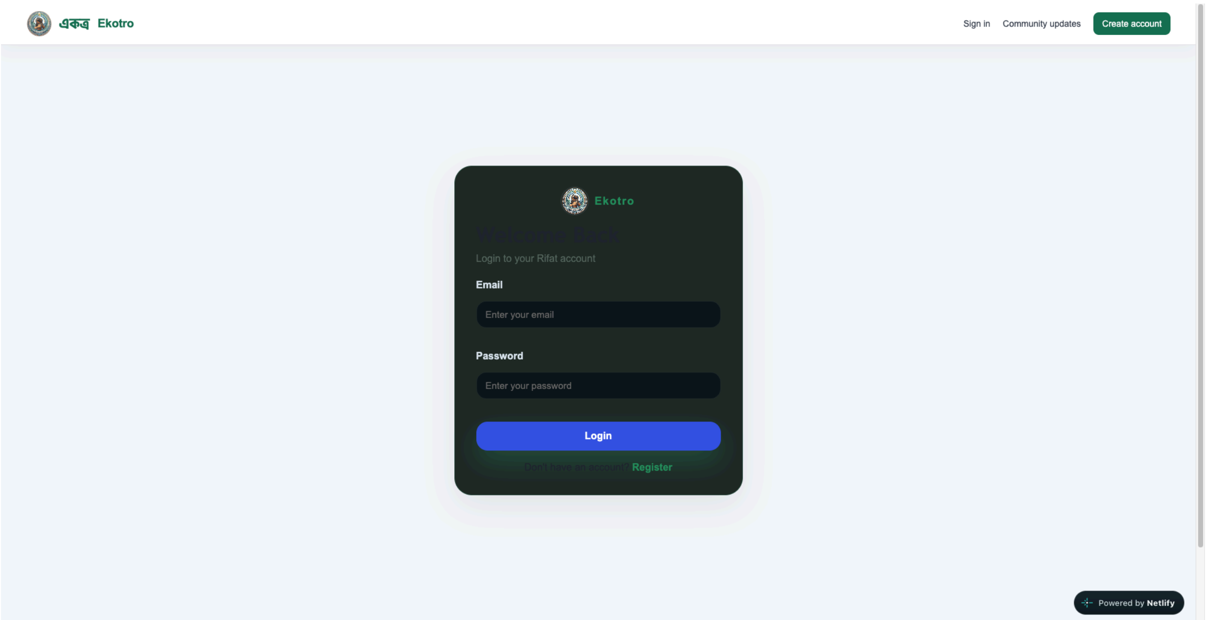


Figure: Login page

7.2 Submit a Complaint

1. Select **Report issue**.
2. Enter the issue title, category, description, and ward or area.

3. Click the map to select the exact location, use the current location button, or enter coordinates manually.
4. Add optional image evidence.
5. Select **Submit community report**.
6. Review the created complaint detail page.

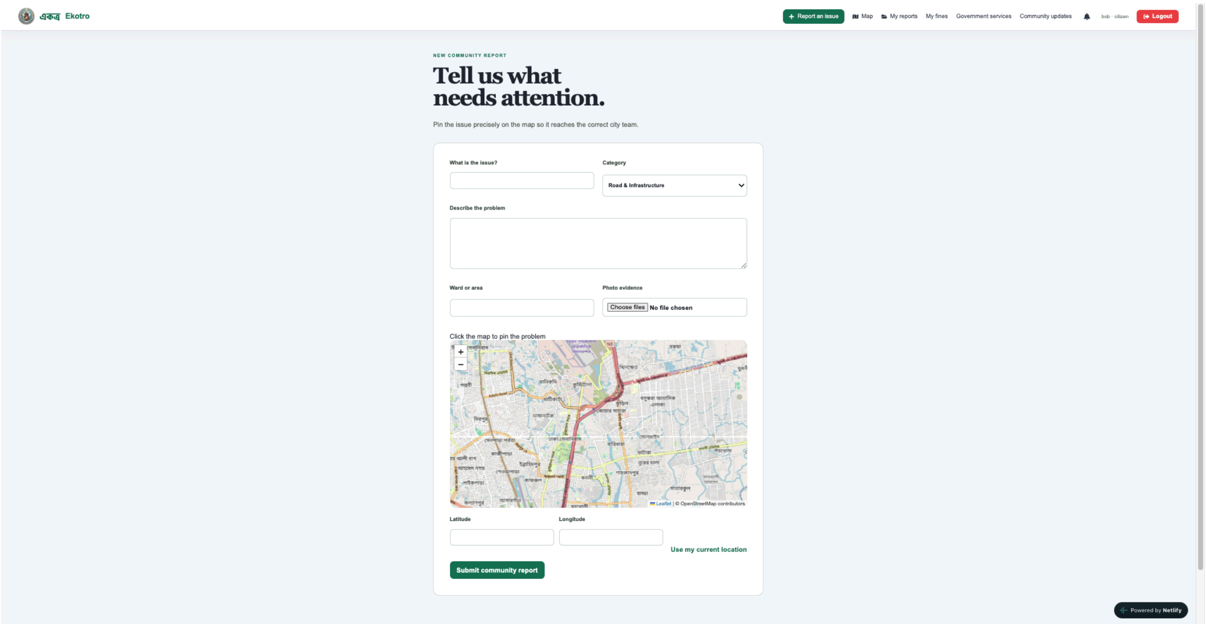


Figure: Complaint submission form

7.3 Browse, Search, Vote, and Comment

1. Open the home feed to browse public complaints.
2. Search or filter by category, status, date, or priority.
3. Open a complaint for its location, timeline, evidence, and ledger.
4. Use the vote/support control to indicate community priority.
5. Add a public comment or reply to an existing comment.
6. Watch for live activity and notification updates.

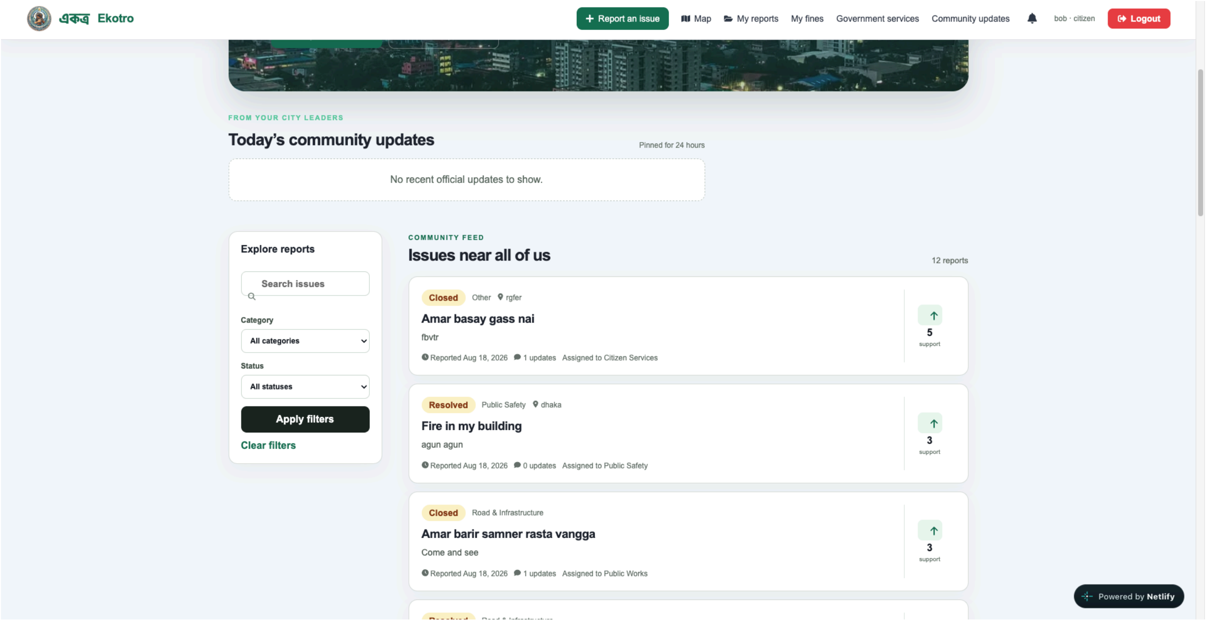


Figure: Complaint feed with filters

একত্র Ekotro

+ Report an issue · Map · My reports · Government services · Community updates · Mayor dashboard · Mayor, Dhaka South City Corporation · mayor · Logout

← All community reports

Closed · Other · Citizen Services

Amar basay gass nai

fbvtr

FIELD WORK EVIDENCE

3 submitted

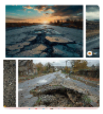
Before and after the repair

Masud
Sep 6, 2026
Road repaired successfully and verified.

Submitted

Before work

After work



Masud
Aug 18, 2026
asnfkf

Submitted

Before work

After work

Masud
Aug 18, 2026
done

Verified

Before work

After work

Officer verification: Work evidence reviewed on site.

rgfar · Reported Aug 18, 2026

ISSUE LOCATION

Public activity

15 events

- **Completion Report Uploaded**
  Before-and-after completion report uploaded.
  Sep 6, 2026
- **Complaint Upvoted**
  A citizen supported this complaint.
  Sep 1, 2026
- **Complaint Upvoted**
  A citizen supported this complaint.
  Sep 1, 2026
- **Completion Report Uploaded**
  Before-and-after completion report uploaded.
  Aug 18, 2026
- **Officer Close**
  This case has been closed. Please submit a new report if the issue returns.
  Aug 18, 2026

Show 10 more activities

CONVERSATION

Public updates

1 posts

Share a useful update with your community...

Post update

M **Mayor, Dhaka South City Corporation**
Mayor · Aug 18, 2026
this should be fixed

Reply to this comment

Reply

PRIVATE CHANNEL

Reporter & authority chat

Only the reporter and assigned municipal authority can view this conversation.

Start the secure case conversation.

Write a private message...

Send

↑

5

people support this issue

Support this issue

One verified support per account.

Issue details

Department
Citizen Services

Priority
Critical

Current state
Closed

Figure: Complaint detail and comments

7.4 Edit or Delete a Complaint

1. Open **My Complaints**.
2. Choose an unassigned complaint.
3. Select **Edit** to change its information or **Delete** to remove it.
4. Once assigned to a department, the complaint should no longer be editable by the citizen.

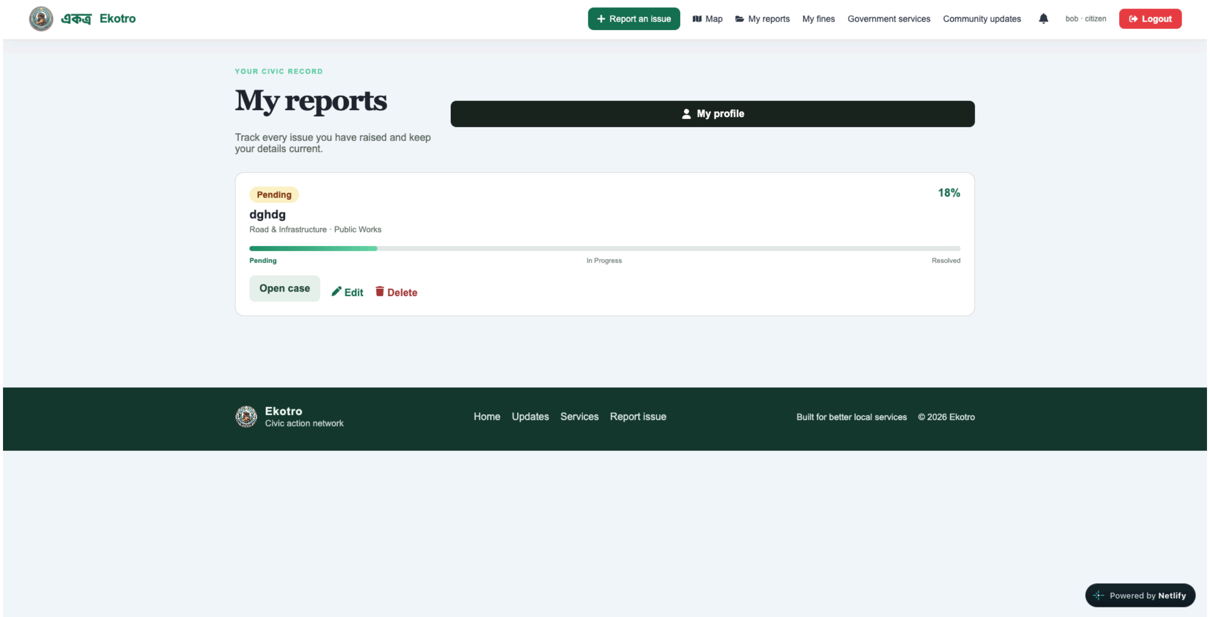


Figure: My Complaints page

7.5 Use Government Services

1. Open **Government Services**.
2. Select a service such as passport, driving license, or birth certificate.
3. Submit the request and description.
4. Open the personal request list to track Pending, Processing, Approved, Rejected, or Completed status.

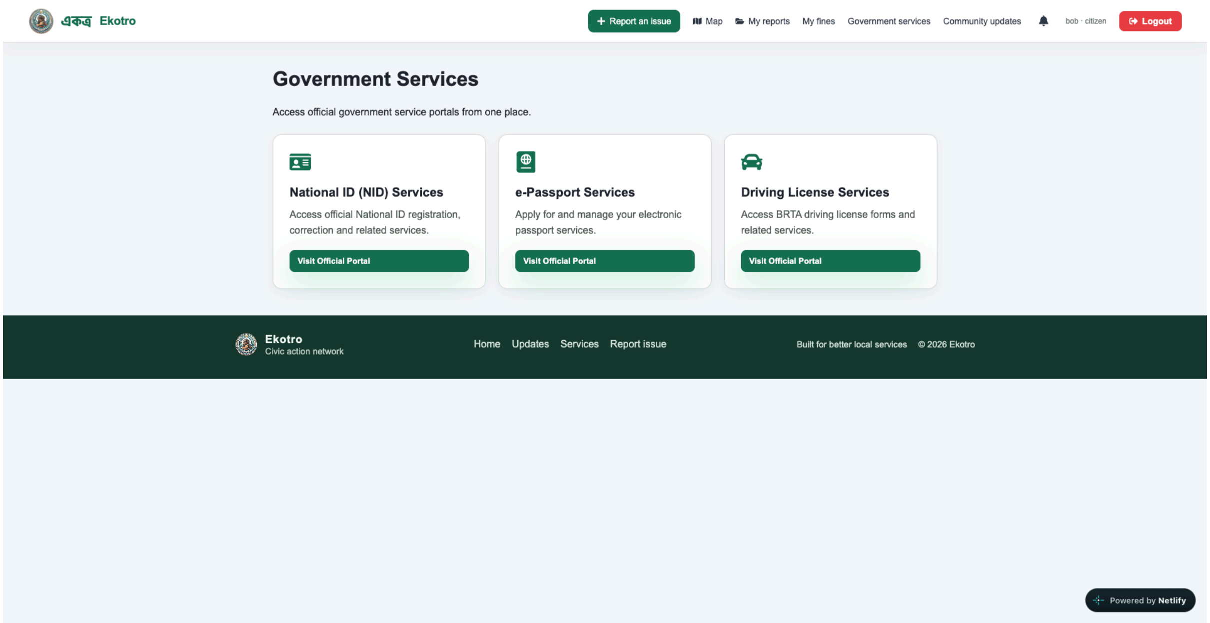


Figure: Government Services page

7.6 Officer, Mayor, Admin, and Field Worker Workflows

- **Officer:** open the officer dashboard, review cases, assign a crew, hold or release a complex case, and verify completion reports.
- **Mayor/Councillor:** publish official updates or announcements for citizens.
- **Field worker:** open an assignment, post progress, and submit before-and-after evidence.
- **Administrator:** inspect analytics, system activity, SLA breaches, and complaint hotspots.
- **Police officer:** create digital fines from the police dashboard and inspect issued fines.
- **Citizen:** open **My Fines** to view received fines and available review/dispute information.

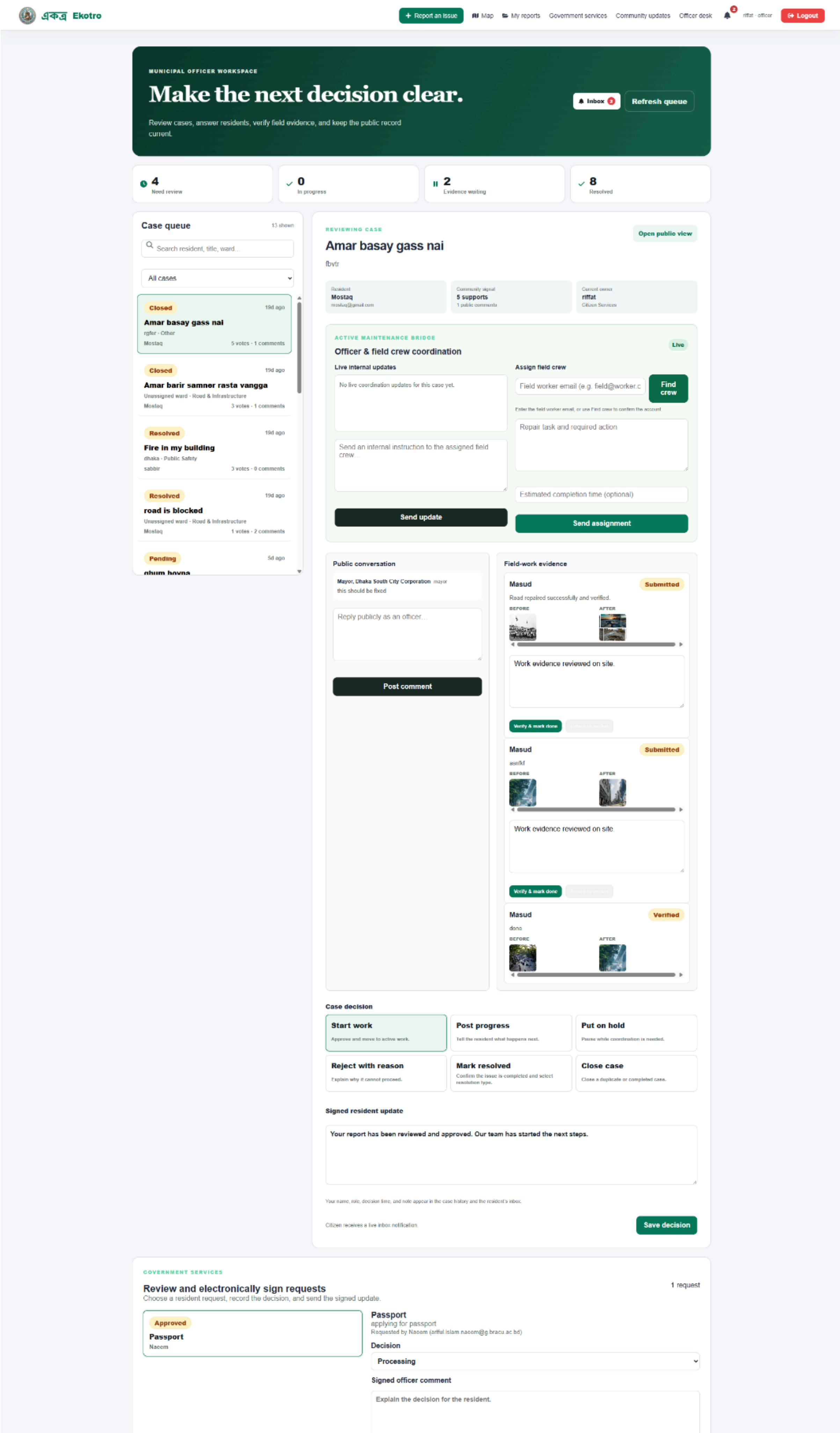

একত্র Ekotro
+ Report an issue
Map
My reports
Government services
Community updates
Officer desk
2
riffat · officer
Logout
MUNICIPAL OFFICER WORKSPACE
Make the next decision clear.
Review cases, answer residents, verify field evidence, and keep the public record current.
Inbox 2
Refresh queue
4 Need review
0 In progress
2 Evidence waiting
8 Resolved
Case queue
13 shown
Search resident, title, ward...
All cases
Closed 19d ago
Amar basay gass nai
rgfer · Other
Mostaq 5 votes · 1 comments
Closed 19d ago
Amar barir samner rasta vangga
Unassigned ward · Road & Infrastructure
Mostaq 3 votes · 1 comments
Resolved 19d ago
Fire in my building
dhaka · Public Safety
sabbir 3 votes · 0 comments
Resolved 19d ago
road is blocked
Unassigned ward · Road & Infrastructure
Mostaq 1 votes · 2 comments
Pending 5d ago
ghum hoyna
REVIEWING CASE
Open public view
Amar basay gass nai
fbvtr
Resident
Mostaq
mostaq@gmail.com
Community signal
5 supports
1 public comments
Current owner
riffat
Citizen Services
ACTIVE MAINTENANCE BRIDGE
Officer & field crew coordination
Live
Live internal updates
No live coordination updates for this case yet.
Send an internal instruction to the assigned field crew...
Send update
Assign field crew
Field worker email (e.g. field@worker.c
Find crew
Enter the field worker email, or use Find crew to confirm the account
Repair task and required action
Estimated completion time (optional)
Send assignment
Public conversation
Mayor, Dhaka South City Corporation mayor
this should be fixed
Reply publicly as an officer...
Post comment
Field-work evidence
Masud
Submitted
Road repaired successfully and verified.
BEFORE
AFTER
Work evidence reviewed on site.
Verify & mark done
Masud
Submitted
asnfkf
BEFORE
AFTER
Work evidence reviewed on site.
Verify & mark done
Masud
Verified
done
BEFORE
AFTER
Case decision
Start work
Approve and move to active work.
Post progress
Tell the resident what happens next.
Put on hold
Pause while coordination is needed.
Reject with reason
Explain why it cannot proceed.
Mark resolved
Confirm the issue is completed and select resolution type.
Close case
Close a duplicate or completed case.
Signed resident update
Your report has been reviewed and approved. Our team has started the next steps.
Your name, role, decision time, and note appear in the case history and the resident's inbox.
Citizen receives a live inbox notification.
Save decision
GOVERNMENT SERVICES
Review and electronically sign requests
1 request
Choose a resident request, record the decision, and send the signed update.
Approved
Passport
Naeem
Passport
applying for passport
Requested by Naeem (ariful.islam.naeem@g.bracu.ac.bd)
Decision
Processing
Signed officer comment
Explain the decision for the resident.

Figure: Officer dashboard

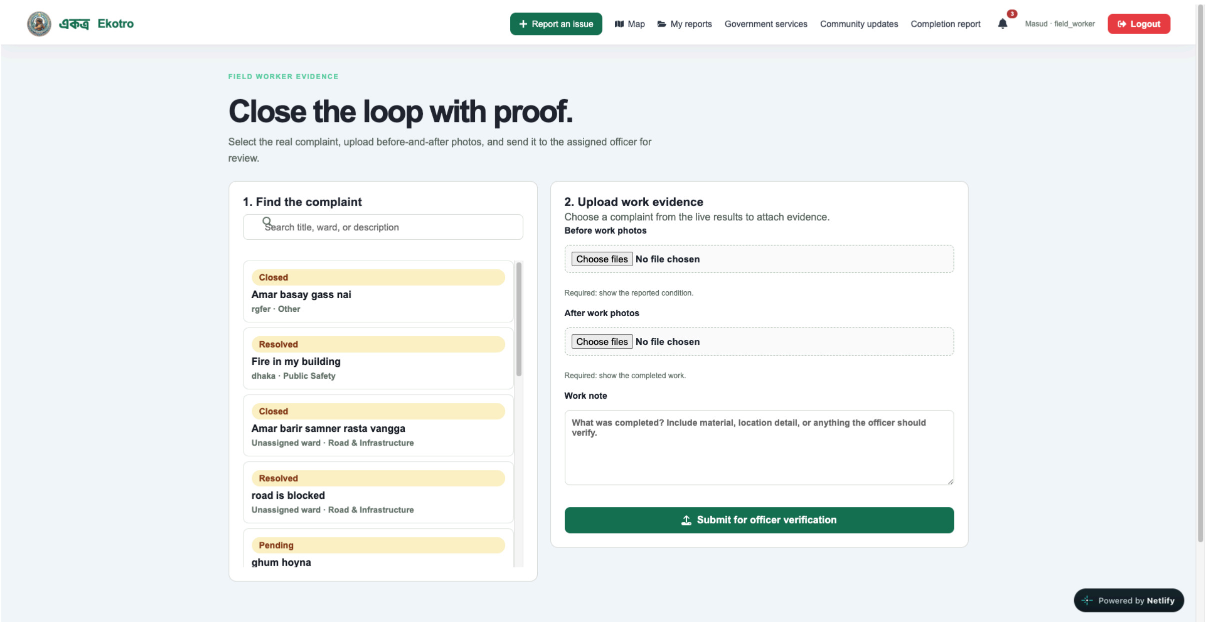


Figure: Field completion report

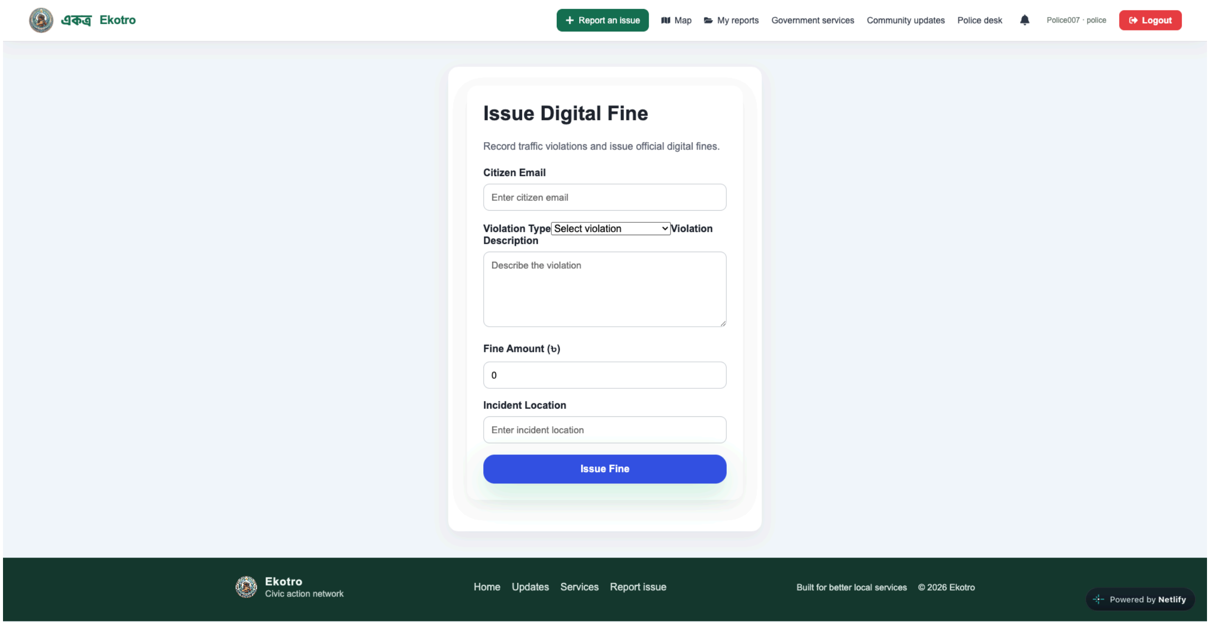


Figure: Police fine form

8. Performance and Network Analysis

Metric	Mobile result	Desktop result
Performance score	70	80
Accessibility score	96	96
Best Practices score	96	96

Metric	Mobile result	Desktop result
SEO score	92	92
First Contentful Paint	2.1 s	1.0 s
Largest Contentful Paint	4.8 s	1.9 s
Total Blocking Time	0 ms	0 ms
Cumulative Layout Shift	0.004	0.065

Interpretation

The main network dependencies are the Netlify frontend bundle, the Render API, MongoDB accessed through the API, Cloudinary image delivery, Socket.IO connections, and OpenStreetMap map tiles. A cold Render instance may increase the initial API response time. Large images are sent through Cloudinary with automatic quality optimization, reducing delivery cost and improving loading compared with serving original files from the API server.

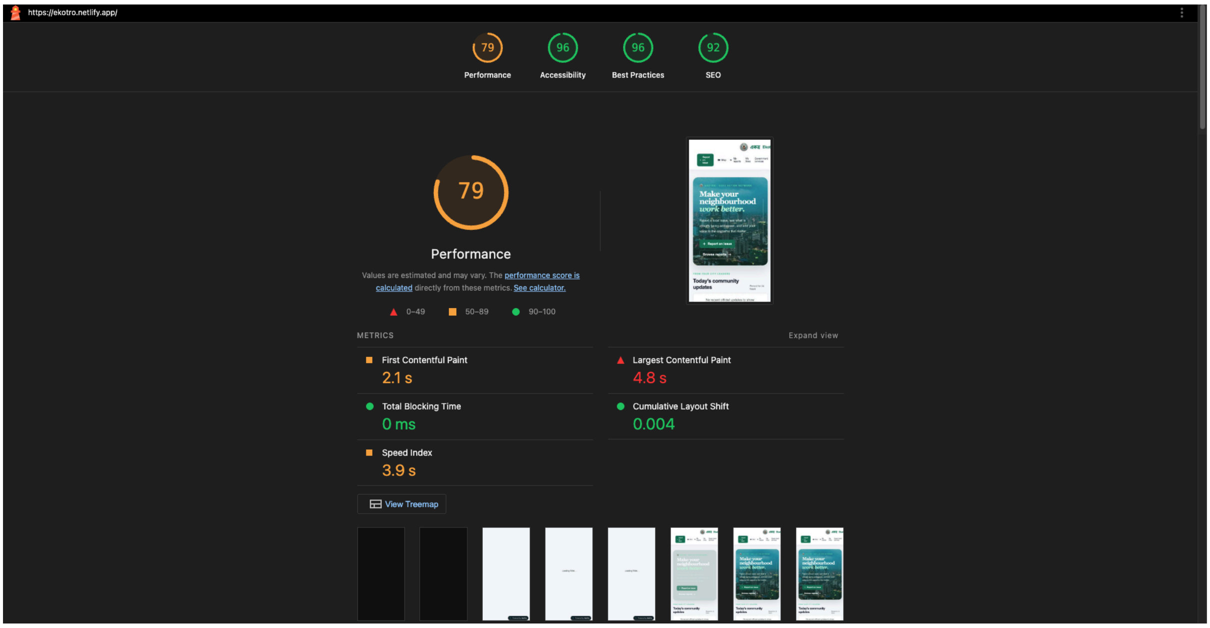


Figure: Lighthouse mobile report

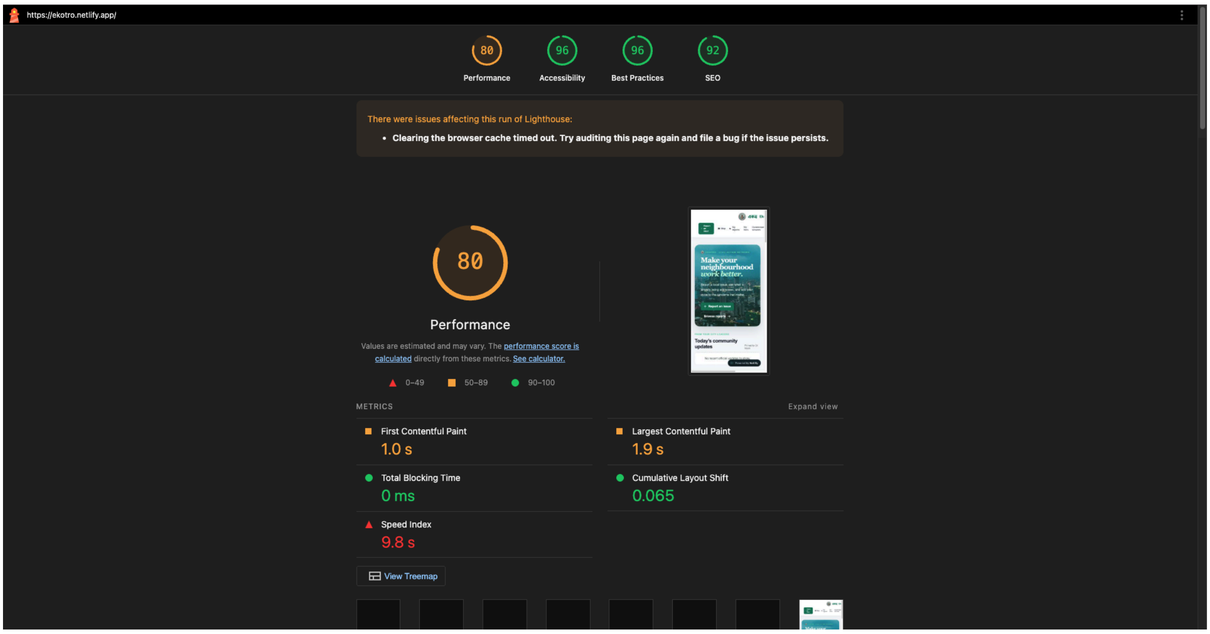


Figure: Lighthouse desktop report

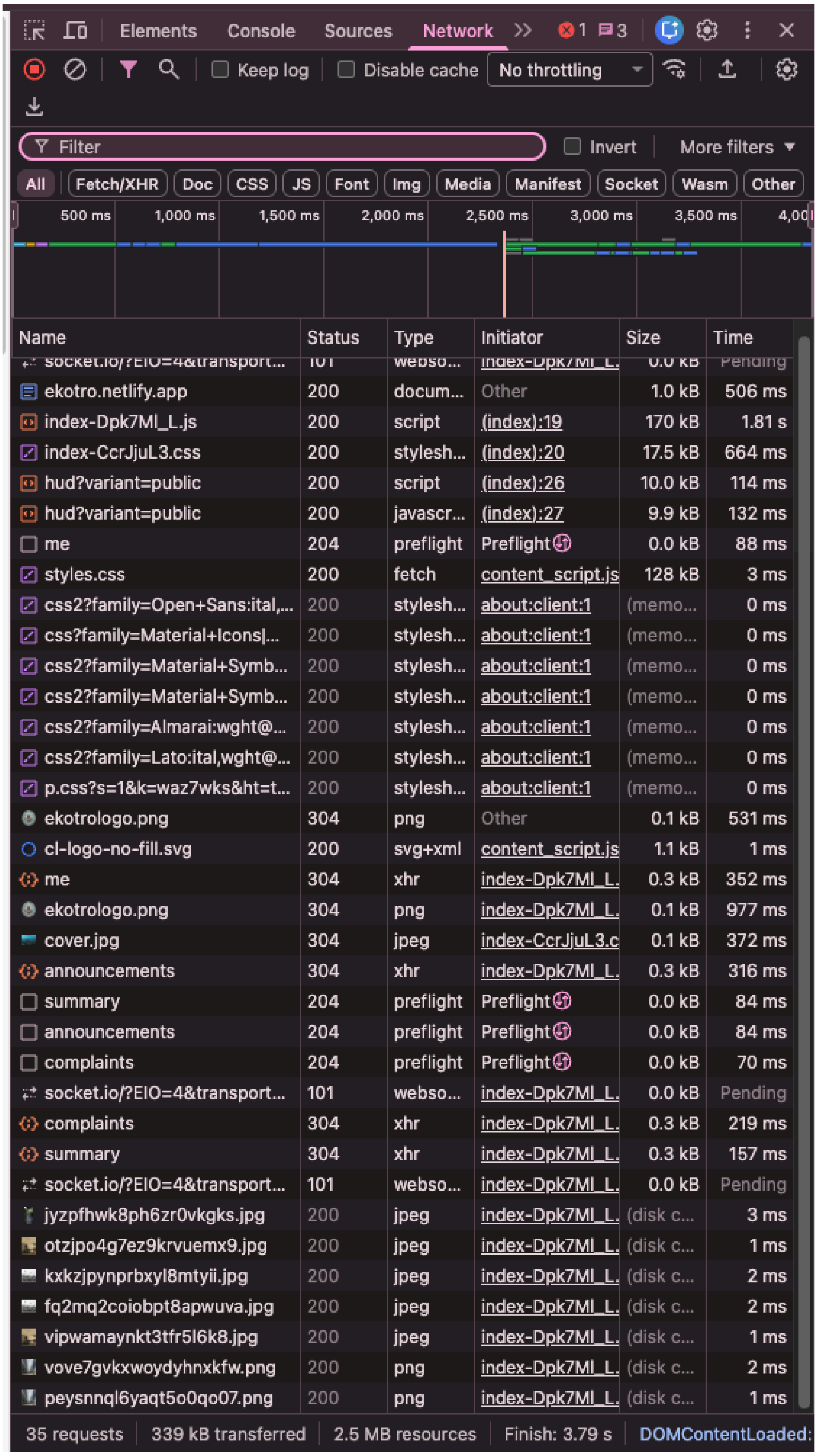
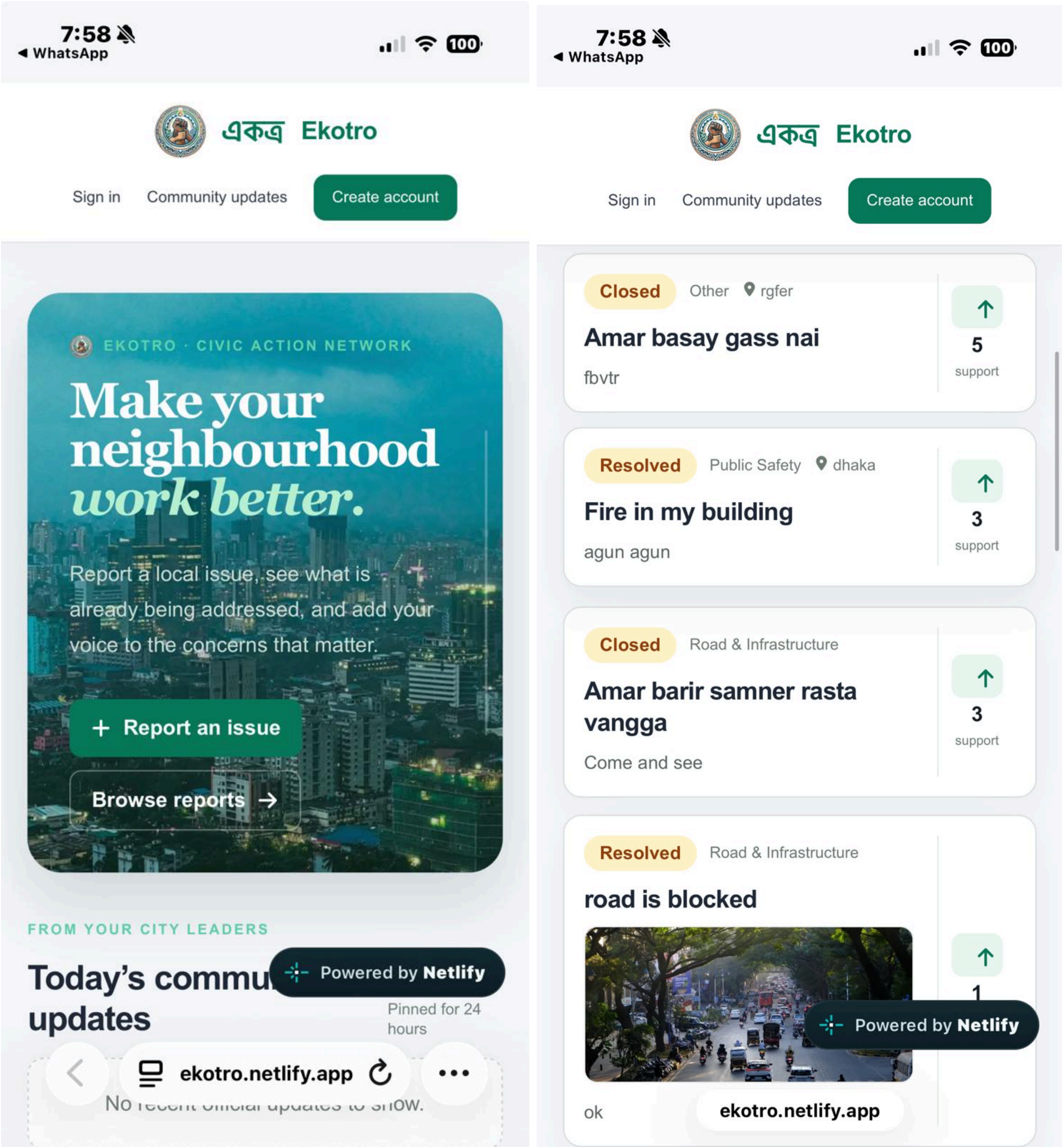


Figure: Browser Network tab showing Ekotro Network Analysis Report



Figures: Ekotro at a mobile viewport

Responsiveness Checklist

- Navigation remains usable on narrow screens.
- Complaint forms stack fields instead of forcing horizontal scrolling.
- Map and complaint cards use responsive widths.
- Buttons and form controls remain accessible on touch devices.
- Dashboard content can be scanned without overlapping labels or controls.

9. Github Repo [Public] Link

Public repository: https://github.com/rifff-fin/471gg

The repository contains the Express/Mongoose backend at the root and the React/Vite frontend in the frontend directory.

10. Link of Deployed Project

- **Frontend:** https://ekotro.netlify.app/
- **Backend:** https://ekotrobackend.onrender.com/

The backend root endpoint can be checked at https://ekotrobackend.onrender.com/. A successful response indicates that the Express service is running.

11. Individual Contribution

Group member - 01	
Name: Ariful Islam Naeem	Student ID:24141141
Functional Requirements which are developed by this member:	
1.Citizens can edit, update, or delete their own complaints before assignment.	
2.Leaflet and MongoDB geospatial indexing for complaint location and spatial filtering.	
3.Field workers can upload completion reports with before-and-after images.	
4.Police officers can issue digital fines with violation details and evidence.	

Group member - 02	
Name: Samara Shahjeen Huq	Student ID:23301214
Functional Requirements which are developed by this member:	
1.Citizens can browse, search, and filter public complaints.	
2.Citizens can comment on public complaints and reply to comments.	
3.Citizens can submit government service requests through a unified portal.	
4.Citizens can view fines and follow review or dispute decisions.	

Group member - 03	
Name: Abdullah Al Rifat	Student ID:22201979

Functional Requirements which are developed by this member:
1.Citizens can submit civic complaints with required details.
2.Socket.IO real-time public comment and activity updates.
3.Councillors and mayors can publish official responses and announcements.
4.Authorized officers can review requests and assign officer roles.
5.Cloudinary optimization and delivery for before-and-after evidence.
6.Administrator dashboard for system statistics and analytics.

Group member - 04 (DROPPED THE COURSE AND HIS PARTS ARE DONE BY ABDULLAH AL RIFAT)	
Name: S M Sabbir Haque Emon	Student ID:23101321
Functional Requirements which are developed by this member:	
1.Cloudinary integration for complaint image hosting and compression.	
2.Citizens can support existing complaints using votes.	
3.Socket.IO internal coordination between citizens, officers, and field crews.	
4.HELD_PENDING complaint state with mandatory rationale and public ledger entry.	

12. References

1. React Documentation: https://react.dev/
2. Vite Documentation: https://vite.dev/
3. Express Documentation: https://expressjs.com/
4. Mongoose Documentation: https://mongoosejs.com/docs/
5. MongoDB Geospatial Queries: https://www.mongodb.com/docs/manual/geospatial-queries/
6. Cloudinary Node.js Documentation: https://cloudinary.com/documentation/node_integration
7. Socket.IO Documentation: https://socket.io/docs/v4/
8. Leaflet Documentation: https://leafletjs.com/
9. OpenStreetMap: https://www.openstreetmap.org/
10. Google Chrome Lighthouse Documentation: https://developer.chrome.com/docs/lighthouse/
11. Ekotro source repository: https://github.com/rifff-fin/471gg
12. Ekotro deployed application: https://ekotro.netlify.app/